

## SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN  
COURSE CODE: CSA1110

---

### COURSE OUTCOME COVERED

**CO4:**

implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (BL5)

*Assessment Tool Weightage: 30%*

---

QUESTIONS: 6

Total Marks:30

Annexure A

Reverse Engineering Task

---

**Code of Conduct:**

I \_\_\_\_ Delisha A K \_\_\_\_\_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.  
I understand that any violation of academic integrity rules will result in disciplinary action.

**Signature of the student:**

# 1. Legacy E-Learning Management System

## 1. Understanding of Existing System

The legacy E-Learning Management System contains major functions for student registration, course management, content delivery, assignment evaluation and result generation. The main architectural problem is that user-interface code and business rules are tightly coupled. Consequently, a change in a screen can require changes in business logic, and changes in rules can affect several UI modules.

## 2. Reverse Engineering and Decomposition

- Student: stores student identity, contact and registration information.
- Course: represents course details, instructor information and course status.
- Enrollment: associates a student with a course and maintains enrollment status.
- LearningContent: represents lectures, videos, documents and other learning resources.
- Assignment: represents an assessment with instructions, deadline and marks.
- Submission: records a student's submitted work and submission status.
- Evaluation: applies assessment rules and records marks/feedback.
- Result: represents calculated course performance.
- UserAccount/AuthService: manages authentication and authorization.
- CourseService, EnrollmentService, EvaluationService and ResultService: contain business operations.
- StudentRepository, CourseRepository, SubmissionRepository and ResultRepository: isolate persistence.
- UI/View layer: presents screens and collects user input.

## 3. Classes and Relationships

- Student 1..\* Enrollment; Enrollment \*..1 Course.
- Course 1..\* LearningContent and Course 1..\* Assignment.
- Assignment 1..\* Submission; Submission belongs to one Student.
- Submission is processed by Evaluation; Evaluation contributes to Result.
- Service classes depend on repository interfaces rather than concrete database implementations.

## 4. Reconstructed Layered Architecture

- Presentation Layer → Service/Business Layer → Domain Model → Repository/Data Access Layer → OODBMS.
- The Presentation Layer contains screens/controllers only.
- The Service Layer coordinates registration, enrollment, evaluation and result-generation use cases.

- The Domain Layer contains cohesive business objects and rules.
- The Repository Layer handles persistence and OODBMS access.

## 5. Design Pattern / OOP Redesign

Use the Service Layer and Repository/Data Access pattern. A Controller accepts the request, invokes the relevant service, the service applies business rules and uses repository interfaces, and repositories perform OODBMS operations. This reconstructs the system into separable, testable object-oriented components.

## 6. Improvement Against the Rubric

- High cohesion: each class has one focused responsibility.
- Low coupling: UI does not directly execute business rules or database operations.
- Maintainability: changes are localized to the appropriate layer.
- Scalability: new content types, assessment types and reporting functions can be added without changing unrelated modules.
- Testability: services can be tested using mock repository interfaces.

## 7. Reconstruction Summary / UML View

**UML-style reconstruction:** Student → Enrollment → Course → {LearningContent, Assignment}; Assignment → Submission → Evaluation → Result; Controller → Services → Repositories → OODBMS.

Rubric alignment: This answer explicitly covers system understanding, detailed decomposition, component identification, logical reconstruction, and structured documentation, targeting the Exemplary (4) descriptors.

## 2. Legacy Customer Support System

### 1. Understanding of Existing System

The legacy customer-support application handles email, chatbot, telephone and social-media requests through multiple conditional statements. Each new channel requires modifications in existing modules. This indicates strong dependence on concrete channel logic, poor separation of responsibilities and violation of the Open/Closed Principle.

### 2. Reverse Engineering and Decomposition

- `SupportRequest`: common representation of a customer issue.
- `SupportChannel`: abstraction defining how a request is received and handled.
- `EmailChannel`, `ChatbotChannel`, `TelephoneChannel` and `SocialMediaChannel`: concrete channel implementations.
- `SupportService`: coordinates request processing.
- `Ticket`: stores request status, priority, assignment and resolution.
- `Agent`: represents the support employee handling a ticket.
- `NotificationService`: sends status updates.
- `ChannelFactory`/Dependency Injection configuration: selects or supplies the required channel implementation.

### 3. Classes and Relationships

- Client depends on the `SupportChannel` abstraction.
- Each concrete channel implements `SupportChannel`.
- `SupportService` uses the abstraction and is independent of concrete channel classes.
- `SupportRequest` creates or updates a `Ticket`, which is assigned to an `Agent`.

### 4. Reconstructed Layered Architecture

- Replace the large conditional structure with a polymorphic strategy.
- `SupportChannel` interface defines `receive()`, `validate()` and `route()` operations.
- Concrete channels implement the interface independently.
- `SupportService` accepts a `SupportChannel` and processes requests without checking channel type.
- A new channel such as `MobileAppChannel` can be introduced by implementing the interface and registering it, without changing `SupportService`.

### 5. Design Pattern / OOP Redesign

Use the Strategy behavioral pattern for interchangeable support-channel behavior. Where runtime selection is required, a Factory can create/select the appropriate strategy. This removes repeated if/else or switch logic and makes the system Open for extension but Closed for modification.

### 6. Improvement Against the Rubric

- Flexibility: behavior can be selected through an abstraction.
- Extensibility: new channels do not require changes to the core service.
- Loose coupling: business logic depends on interfaces.
- Maintainability: channel-specific code is isolated.
- Testability: each channel and the support service can be tested independently.

### 7. Reconstruction Summary / UML View

UML-style reconstruction: `SupportService` → `SupportChannel` &lt;&lt;interface&gt;&gt; ← {`EmailChannel`, `ChatbotChannel`, `TelephoneChannel`, `SocialMediaChannel`}; `SupportRequest` → `Ticket` → `Agent`.

Rubric alignment: This answer explicitly covers system understanding, detailed decomposition, component identification, logical reconstruction, and structured documentation, targeting the Exemplary (4) descriptors.

### 3. Legacy Banking Transaction System

#### 1. Understanding of Existing System

The legacy banking system combines account management, transaction validation, fund transfer, loan processing, transaction reporting and database operations in tightly coupled classes. Business methods directly access the database, so domain logic, application logic and persistence are mixed.

#### 2. Reverse Engineering and Decomposition

- Account: account number, holder, balance and account status.
- Customer: customer identity and associated accounts.
- Transaction: amount, type, timestamp and status.
- Loan: loan amount, interest, term and repayment status.
- AccountService: account-related operations.
- TransactionService: validation and transaction processing.
- TransferService: coordinates transfers between accounts.
- LoanService: handles loan processing.
- ReportService: generates transaction reports.
- AccountRepository, TransactionRepository and LoanRepository: persistence abstractions.
- OODBMS/Data Access implementation: performs database operations.

#### 3. Classes and Relationships

- Customer 1..\* Account.
- Account 1..\* Transaction.
- TransferService coordinates source and destination Account objects.
- Loan belongs to a Customer and may be associated with an Account.
- Services depend on repository interfaces, while concrete repositories depend on the OODBMS.

#### 4. Reconstructed Layered Architecture

- Presentation Layer → Application/Service Layer → Domain Layer → Repository Layer → OODBMS.
- Controllers handle requests and responses.
- Services contain use-case orchestration and validation.
- Domain classes represent banking concepts and invariants.
- Repositories isolate persistence operations.

#### 5. Design Pattern / OOP Redesign

Use the Repository/Data Access pattern together with a layered architecture. The service layer coordinates transactions, while repository interfaces provide persistence contracts. Dependency inversion keeps business logic independent of concrete OODBMS classes.

#### 6. Improvement Against the Rubric

- Separation of concerns: database operations are removed from business methods.
- Testability: services can use mock repository implementations.
- Maintainability: persistence changes do not require rewriting domain logic.
- Consistency: validation is centralized in appropriate services/domain objects.
- Scalability: additional reports, transaction types or storage implementations can be added with limited impact.

## 7. Reconstruction Summary / UML View

<b>UML-style reconstruction:</b> Controller → Services → Repository<&lt;interface>&gt; ← OODBMS Repository;  
Customer → Account → Transaction; LoanService → Loan.

Rubric alignment: This answer explicitly covers system understanding, detailed decomposition, component identification, logical reconstruction, and structured documentation, targeting the Exemplary (4) descriptors.

## 4. Legacy Hotel Reservation System

### 1. Understanding of Existing System

The legacy hotel reservation system performs OODBMS operations directly from reservation screens and repeats database queries for rooms, guests, bookings and payments. This creates tight coupling between presentation and persistence, duplicated queries and inconsistent persistence management.

### 2. Reverse Engineering and Decomposition

- Room: room number, type, capacity, price and availability.
- Guest: guest identity and contact information.
- Reservation: booking dates, status, guest and room association.
- Payment: amount, method, date and payment status.
- ReservationController/UI: accepts reservation requests.
- ReservationService: validates availability and coordinates booking.
- RoomRepository, GuestRepository, ReservationRepository and PaymentRepository: persistence abstractions.
- OODBMS adapter/data-access implementation: performs object persistence and retrieval.

### 3. Classes and Relationships

- Guest 1..\* Reservation.
- Room 1..\* Reservation over time, subject to availability.
- Reservation may have one or more Payment records depending on business rules.
- Services depend on repository interfaces instead of direct OODBMS queries.

### 4. Reconstructed Layered Architecture

- Presentation Layer → Reservation/Booking Service → Domain Objects → Repository Interfaces → OODBMS.
- Common database operations are centralized in repositories.
- The same repository can be reused by reservation, cancellation, reporting and administration modules.
- OODBMS-specific persistence details remain inside the data-access implementation.

### 5. Design Pattern / OOP Redesign

Use DAO/Repository patterns to encapsulate persistence. The Repository provides domain-oriented operations such as findAvailableRooms(), saveReservation() and findGuest(), while the service layer handles reservation rules. This produces a clean integration boundary with the OODBMS.

### 6. Improvement Against the Rubric

- Modularity: screens no longer know persistence details.
- Reusability: common room/guest/booking queries exist in one place.
- Reduced duplication: repeated database logic is centralized.
- Maintainability: OODBMS changes are isolated.
- Testability: reservation rules can be tested without a real database.

### 7. Reconstruction Summary / UML View

<b>UML-style reconstruction:</b> ReservationController → ReservationService → {RoomRepository, GuestRepository, ReservationRepository, PaymentRepository} → OODBMS; Guest → Reservation ← Room.

Rubric alignment: This answer explicitly covers system understanding, detailed decomposition, component identification, logical reconstruction, and structured documentation, targeting the Exemplary (4) descriptors.



## 5. Legacy Transportation Management System

### 1. Understanding of Existing System

The transportation system supports buses, trains, taxis and rental vehicles, but object-creation logic is duplicated across several modules. The duplication makes the code difficult to extend when a new transportation service is introduced.

### 2. Reverse Engineering and Decomposition

- Transport: common abstraction containing shared transportation operations such as start(), stop(), getCapacity() and calculateFare().
- Bus, Train, Taxi and RentalVehicle: concrete transportation classes.
- TransportFactory: creates the appropriate Transport object.
- Booking/TransportService: uses the Transport abstraction without knowing the concrete class.
- Optional TransportFactory subclasses can encapsulate specialized creation rules.

### 3. Classes and Relationships

- Bus, Train, Taxi and RentalVehicle implement or inherit from Transport.
- Client/TransportService depends on Transport rather than concrete classes.
- TransportFactory creates the required concrete object.
- The common abstraction allows shared client logic to be reused across transportation types.

### 4. Reconstructed Layered Architecture

- Client → TransportFactory → Transport abstraction → Bus/Train/Taxi/RentalVehicle.
- For Factory Method, a creator defines the factory operation and specialized creators create particular transport objects.
- If multiple related transport families are introduced, an Abstract Factory can provide a family of related objects.

### 5. Design Pattern / OOP Redesign

Apply the Factory Method pattern to centralize transportation-object creation. The client requests a Transport from the factory and works with the common abstraction. This removes duplicated new-object logic and supports future transportation types.

### 6. Improvement Against the Rubric

- Extensibility: a new service such as Metro can be added with a new concrete class and factory logic.
- Scalability: object creation remains centralized.
- Code reuse: clients use the same abstraction and do not duplicate construction code.
- Maintainability: creation rules are separated from business operations.
- Loose coupling: client code does not depend on concrete transportation classes.

### 7. Reconstruction Summary / UML View

**UML-style reconstruction:** TransportService → TransportFactory → Transport<interface> ← {Bus, Train, Taxi, RentalVehicle}.

Rubric alignment: This answer explicitly covers system understanding, detailed decomposition, component identification, logical reconstruction, and structured documentation, targeting the Exemplary (4) descriptors.

## 6. Legacy Healthcare Monitoring System

### 1. Understanding of Existing System

The healthcare monitoring system collects patient information, processes sensor readings, generates alerts, stores medical records and produces reports. Several classes perform multiple unrelated tasks and directly depend on concrete monitoring devices and storage components. These characteristics indicate violations of SRP, OCP and DIP.

### 2. Reverse Engineering and Decomposition

- Patient: patient identity and healthcare profile.
- SensorReading: timestamped measurement with value, type and source.
- MonitoringDevice: abstraction for obtaining readings.
- HeartRateMonitor, TemperatureMonitor and other devices: concrete device implementations.
- ReadingProcessor: validates and interprets readings.
- AlertService: determines and creates alerts.
- MedicalRecord: stores clinically relevant monitoring information.
- MedicalRecordRepository: persistence abstraction.
- ReportService: produces patient monitoring reports.
- NotificationService: delivers alerts.
- MonitoringService: coordinates the monitoring workflow.

### 3. Classes and Relationships

- MonitoringDevice is an interface implemented by concrete devices.
- MonitoringService depends on MonitoringDevice abstraction.
- ReadingProcessor consumes SensorReading objects.
- AlertService creates alerts based on processed readings.
- MedicalRecordRepository abstracts storage.
- ReportService reads required information through the repository rather than directly accessing storage.

### 4. Reconstructed Layered Architecture

- SRP: separate patient management, reading processing, alert generation, persistence and reporting into focused classes.
- OCP: define interfaces for monitoring devices so new device types can be added without modifying monitoring logic.
- DIP: services depend on MonitoringDevice and MedicalRecordRepository abstractions, not concrete classes.
- Use Strategy where different reading-analysis algorithms may be selected, and Observer where multiple listeners need notification when an alert/event occurs.

### 5. Design Pattern / OOP Redesign

Reconstruct the system around interfaces and small cohesive services. MonitoringService receives readings from a MonitoringDevice abstraction, passes them to ReadingProcessor, invokes AlertService when thresholds/rules require an alert, stores relevant information through MedicalRecordRepository, and publishes notifications. This directly addresses SRP, OCP and DIP and produces a maintainable layered design.

### 6. Improvement Against the Rubric

- Cohesion improves because each class has a focused responsibility.
- Coupling decreases because abstractions separate services from devices and storage.
- Flexibility improves because new devices and analysis strategies can be plugged in.

- Testability improves because fake device/repository implementations can be injected.
- Maintainability improves because changes remain localized.

## 7. Reconstruction Summary / UML View

**UML-style reconstruction:** MonitoringService → MonitoringDevice<<interface>> ← {HeartRateMonitor, TemperatureMonitor}; → ReadingProcessor → AlertService → NotificationService; → MedicalRecordRepository → OODBMS.

Rubric alignment: This answer explicitly covers system understanding, detailed decomposition, component identification, logical reconstruction, and structured documentation, targeting the Exemplary (4) descriptors.